Contents

1 世界模型 **3**
1.1 pandora . . . 4
1.2 WHALE . . . 4
1.3 genie 2 . . . 4
1.4 NWM . . . 4

2 语言物理学 **4**

3 LLM+math **4**
3.1 mathscale . . . 4
3.2 AlphaGeometry . . . 5
3.3 AlphaProof & AlphaGeometry 2 . . . 5
3.4 WE-Math 基准 . . . 5
3.5 case-based or rule-based . . . 5

4 LLM 常见难题 **5**
4.1 重复生成 . . . 5
4.2 幻觉 . . . 5
4.2.1 综述 . . . 5
4.2.2 语义熵 . . . 5
4.3 知识冲突 . . . 5
4.4 llm+db . . . 5
4.5 Zilliz . . . 6
4.6 记忆能力 . . . 6
4.6.1 reasoning . . . 6
4.6.2 memorizing . . . 6
4.7 越狱 . . . 6
4.8 LLM compiler . . . 6
4.9 ProLong . . . 6
4.10 白化 . . . 6
4.11 蒸馏 . . . 7
4.12 证明者-验证者博弈 . . . 7
4.13 道德风险 . . . 7
4.14 选择性偏差 . . . 7
4.15 lost in the middle . . . 7
4.16 reasoning boundary . . . 7
4.17 语言 $\neq$ 思维 . . . 7

5 多智能体 **7**
5.1 JAT . . . 8
5.2 ReadAgent . . . 8
5.3 多模态 agent . . . 8
5.4 OpenDevin . . . 9
5.5 autogpt . . . 9
5.6 DAAG . . . 9
5.7 VARP . . . 9
5.8 MARL . . . 9
5.9 MMRole . . . 9
5.10 Swarm . . . 9
5.11 agent-as-a-judge . . . 9
5.12 Hammer . . . 10
5.13 AgentOccam . . . 10
5.14 苏格拉底学习 . . . 10

5.15 从个人模拟到社会模拟 . . . 10
5.16 MetaGPT . . . 10
5.17 insight-V . . . 10
5.18 世界经济论坛的 agent 报告 . . . 10
5.19 Claude: building effective agents . . . 10
5.19.1 基石: augmented LLM . . . 11
5.19.2 workflow . . . 11
5.19.3 Agents . . . 13

6 一些其他比较重要的工作 14
6.1 几篇出现频率比较高的论文 . . . 14
6.2 Anthropic 的一些工作 . . . 14

7 个性化搜索 14
7.1 q-i 双塔的改进 . . . 15
7.1.1 引入 location+social . . . 15
7.1.2 引入用户行为序列 . . . 15
7.1.3 三塔 +gnn 邻居 +mtl . . . 16

8 RAG 17
8.1 RankRAG . . . 17
8.2 graphRAG . . . 18
8.3 RAGChecker . . . 18
8.4 TAG . . . 18
8.5 Storm . . . 18
8.6 Block-atttention RAG . . . 18
8.7 2024 rags . . . 18

9 LLM 模型融合 18

10 LLM auto-ml 19
10.1 LLaMA-NAS . . . 19
10.2 SELA . . . 19

11 prompt engineering 19
11.1 APE . . . 20
11.2 PAS . . . 20
11.3 ell . . . 20

12 可解释 AI 20
12.1 综述 . . . 21
12.2 TransformerLens . . . 21
12.3 ecco . . . 21
12.4 interpretability in the wild . . . 21
12.5 activation engineering . . . 21
12.6 representation engineering . . . 21
12.7 transformer-debugger . . . 21
12.8 painter . . . 21
12.9 transformer explainer . . . 21
12.10superposition . . . 21
12.113Blue1Brown . . . 22
12.12Monitor . . . 22

13 Lifelong learning of LLM 22

14 自省 **22**

15 具身智能 **22**
15.1 ReKep . . . 22
15.2 GR-2 . . . 22
15.3 RDT-1B . . . 22
15.4 HIL-SERL . . . 23
15.5 Genesis . . . 23
15.6 language of motion . . . 23

16 语音 **23**
16.1 Mini-Omni . . . 23
16.2 Illuminate . . . 23

17 LLM+ 芯片设计 **23**

18 其他 **23**
18.1 安全性 . . . 23
18.2 time-LLM . . . 23
18.3 长尾 . . . 24
18.4 文本匹配效果还行的模型 . . . 24
18.5 本地知识库 . . . 24
18.6 llm 应用合辑 . . . 24
18.7 swiftsage . . . 24
18.8 达摩院大模型技术交流 . . . 24
18.9 回译 . . . 24
18.10nan 问题 . . . 24
18.11时间序列 . . . 25
18.12人脑 . . . 25
18.13LLM for 算法设计 . . . 25
18.14深度生成模型课程 . . . 25
18.15其他 . . . 25

19 一些记录 **26**
19.1 打印模型参数量 . . . 26
19.2 往现有 tokenizer 里加一些特殊 token . . . 26
19.3 python 的读写锁 . . . 27
19.4 pytorch 的显存泄露 . . . 27
19.5 torch profiling . . . 27
19.6 显存泄露排查 . . . 27
19.7 各型号 gpu 对比 . . . 30
19.8 查看 python 的栈 . . . 30
19.9 国内的 huggingface 模型下载地址 . . . 31
19.10一些报错的解法 . . . 31
19.10.1 flash-attention2 . . . 31

20 GPU 机型对比 **31**

1 世界模型

怒斥 Sora 之后，LeCun 放出「视觉世界模型」论文，揭示 AI 学习物理世界的关键

100 万 token，一次能分析 1 小时 YouTube 视频，「大世界模型」火了

WORLD MODEL ON MILLION-LENGTH VIDEO AND LANGUAGE WITH RINGATTENTION

https://github.com/LargeWorldModel/LWM

Sora 是世界模拟器吗？全球首篇综述全面解析通用世界模型

Is Sora a World Simulator? A Comprehensive Survey on General World Models and Beyond

世界模型也扩散！训练出的智能体竟然不错

Diffusion for World Modeling: Visual Details Matter in Atari

https://github.com/eloialonso/diamond

1.1 pandora

通用世界模型问世：不学习就能生成新领域视频，可实时控制

Pandora : Towards General World Model with Natural Language Actions and Video States

https://github.com/maitrix-org/Pandora

ACL 2024 论文盖棺定论：大语言模型 ≠ 世界模拟器，Yann LeCun：太对了

Can Language Models Serve as Text-Based World Simulators?

1.2 WHALE

WHALE 来了，南大周志华团队做出更强泛化的世界模型

WHALE: TOWARDS GENERALIZABLE AND SCALABLE WORLD MODELS FOR EMBODIED DECISION-MAKING

1.3 genie 2

谷歌世界模型爆发：单张图生成可玩 3D 世界，还要和马斯克一起做 AI 游戏

https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/

1.4 NWM

LeCun 团队新作：在世界模型中导航

Navigation World Models

https://www.amirbar.net/nwm/

2 语言物理学

(toread)

大模型边推理边纠错，有可能做到吗？这是 ICML 爆火的演讲

https://www.bilibili.com/video/BV1Yw4m1k7nH

3 LLM+math

3.1 mathscale

【LLM-数学】MathScale 用于数学推理的指令调优扩展方法

MathScale: Scaling Instruction Tuning for Mathematical Reasoning

3.2 AlphaGeometry

奥数能力金牌级：DeepMind 几何推理模型登上 Nature，代码开源，菲尔兹奖得主点赞提出了 AlphaGeometry

3.3 AlphaProof & AlphaGeometry 2

谷歌 AI 拿下 IMO 奥数银牌，数学推理模型 AlphaProof 面世，强化学习 is so back提出 AlphaProof 和 AlphaGeometry 2

3.4 WE-Math 基准

真相了！大模型解数学题和人类真不一样：死记硬背、知识欠缺明显，GPT-4o 表现最佳

WE-MATH: Does Your Large Multimodal Model Achieve Human-like Mathematical Reasoning?

https://github.com/We-Math/We-Math

https://huggingface.co/datasets/We-Math/We-Math

3.5 case-based or rule-based

ICML 2024 | Transformer 究竟如何推理？基于样例还是基于规则

Case-Based or Rule-Based: How Do Transformers Do the Math?

https://github.com/GraphPKU/Case_or_Rule

4 LLM 常见难题

4.1 重复生成

https://www.zhihu.com/question/616130636

https://mp.weixin.qq.com/s/cSwWapqFhxu9zafzPUeVEw

4.2 幻觉

4.2.1 综述

OpenAI Lilian Weng 万字长文解读 LLM 幻觉：从理解到克服

https://lilianweng.github.io/posts/2024-07-07-hallucination/

4.2.2 语义熵

语义熵识破 LLM 幻觉！牛津大学新研究登 Nature

Detecting hallucinations in large language models using semantic entropy

4.3 知识冲突

深度解析 RAG 大模型知识冲突，清华西湖大学港中文联合发布

Knowledge Conflicts for LLMs: A Survey

4.4 llm+db

长文本杀不死 RAG：SQL+ 向量驱动大模型和大数据新范式，MyScale AI 数据库正式开源

https://github.com/myscale/myscaledb

4.5 Zilliz

向量数据库的中场战事：长期主义者 Zilliz 如何全球突围

4.6 记忆能力

Localizing Paragraph Memorization in Language Models

对应代码：https://github.com/googleinterns/localizing-paragraph-memorization

我们能否定位出语言模型中用于记忆其训练数据中整段文字的权重和机制？

- 尽管记忆现象分布在模型的多个层级和组件中，但记忆段落的梯度在空间上有可辨别的模式，即**在较低模型层级的梯度比非记忆 example 的梯度大**。
- 通过**仅微调高梯度的权重**，可以使模型**遗忘记忆的 example**。
- 定位了一个特别参与段落记忆的**低层注意力头**，它**主要关注**在语料库级单词频率分布中**最不频繁出现的独特、罕见的 token**。
- 总的来说，相较非记忆的续写，记忆续写不仅**更难以遗忘**，也**更难以损坏**。

4.6.1 reasoning

Reasoning or Reciting? Exploring the Capabilities and Limitations of Language Models Through Counterfactual Tasks MIT 的

Do Large Language Models Latently Perform Multi-Hop Reasoning? deepmind 的

How do Language Models Bind Entities in Context? UC berkeley 的，ICLR2024

4.6.2 memorizing

Knowledge Neurons in Pretrained Transformers ACL 2022

Language Modeling Is Compression ICLR 2024 deepmind

Memorization Without Overfitting: Analyzing the Training Dynamics of Large Language Models meta NeurIPS 2022

4.7 越狱

长文本之罪：Claude 团队新越狱技术，Llama 2 到 GPT-4 无一幸免

4.8 LLM compiler

开发者狂喜！Meta 最新发布的 LLM Compiler，实现 77% 自动调优效率

Meta Large Language Model Compiler: Foundation Models of Compiler Optimization

4.9 ProLong

2024 年了，你的长文本训练数据真的够长吗？

Long Context is Not Long at All: A Prospector of Long-Dependency Data for Large Language Models

https://github.com/October2001/ProLong

4.10 白化

在 transformer 领域里，“白化”（whitening）主要是指一种对句子嵌入进行后处理的方法，通过将句子向量的均值变为 0，并将协方差矩阵变为单位矩阵，从而解决句子嵌入中的各向异性问题。这种技术能够提高句子嵌入在语义相似性任务中的表现，并且加快检索速度。

Whitening Sentence Representations for Better Semantics and Faster Retrieval

代码：https://github.com/bojone/BERT-whitening

Transformer Scale Gate for Semantic Segmentation

4.11 蒸馏

Revisiting Knowledge Distillation for Autoregressive Language Models

Meta 开发 System 2 蒸馏技术，Llama 2 对话模型任务准确率接近 100%

Distilling System 2 into System 1

4.12 证明者-验证者博弈

OpenAI 超级对齐团队遗作：两个大模型博弈一番，输出更好懂了

Prover-Verifier Games improve legibility of LLM outputs

参考：Learning to Give Checkable Answers with Prover-Verifier Games

4.13 道德风险

GPT-4o 模仿人类声音，诡异尖叫引 OpenAI 研究员恐慌！32 页技术报告出炉

openai 的报告：GPT-4o System Card

之前 deepmind 也有一个报告The Ethics of Advanced AI Assistants

4.14 选择性偏差

ACL2024| 大模型选择偏差在腾讯广告特征评测上的优化及应用

Strengthened Symbol Binding Makes Large Language Models Reliable Multiple-Choice Selectors

给定一个问题 (question) 及其对应的选项内容 (options)，大模型无法把选项内容 (option content) 和对应的选项标识符 (symbol，特指选项标识 A/B/C/D) 关联到一起。例如，当把正确答案”the president”放到选项 B 时，模型能够正确选择出答案；当我们把正确答案放到 C 时，模型依然选择”B”，即模型偏向于选”B”或者第二个答案，而忽略了正确答案的内容。

4.15 lost in the middle

Lost in the Middle: How Language Models Use Long Contexts

4.16 reasoning boundary

NeurIPS 2024 (Oral) | 如何量化与提升思维链的推理能力边界?

Unlocking the Capabilities of Thought: A Reasoning Boundary Framework to Quantify and Optimize Chain-of-Thought

https://github.com/LightChen233/reasoning-boundary

4.17 语言 ≠ 思维

语言 ≠ 思维，大模型学不了推理：一篇 Nature 让 AI 社区炸锅了

https://www.nature.com/articles/s41586-024-07522-w

5 多智能体

《综述：全新大语言模型驱动的 Agent》——4.5 万字详细解读复旦 NLP 和米哈游最新 Agent Survey

Agent > GPT5? 吴恩达最新演讲：四种 Agent 设计范式（通俗易懂版）

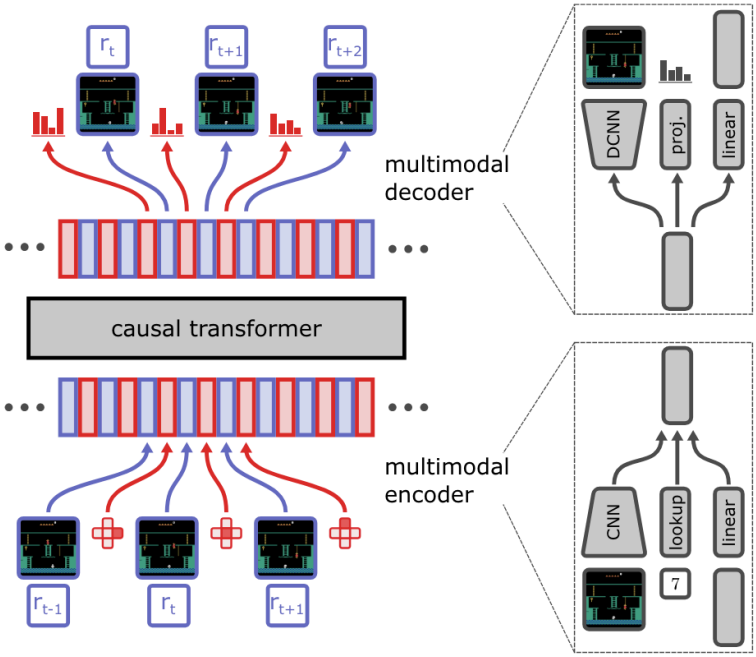
5.1 JAT

告别偏科，能玩转多模态、多任务、多领域的强化智能体终于来了

Jack of All Trades, Master of Some, a Multi-Purpose Transformer Agent

https://github.com/huggingface/jat

https://huggingface.co/datasets/jat-project/jat-dataset



输入的序列元素是 observations, actions, 和 rewards 的交替组合：

$$[\phi(s_0, 0.0), \phi(a_0), \phi(s_1, r_1), \phi(a_1), ...]$$

依据不同输入的数据类型，使用不同网络处理：

- 图像：用 CNN。
- 连续向量：用线性层
- 离散值：用线性投影层

预测任务：根据所有先前的观察和动作嵌入来预测下一个动作嵌入。

序列的构造方法：

- 和文本相关的任务：用 GPT-2 的分词策略，将文本转换为一个整数序列，然后 emb lookup 映射到一个嵌入向量序列。
- 和图像有关的任务：用 ViT，将图像切割成小块后，通过线性层转换为嵌入向量序列。
- 最终再将图像和文本的向量序列拼接在一起，形成一个统一的序列，输入到 Transformer 中。

5.2 ReadAgent

「有效上下文」提升 20 倍！DeepMind 发布 ReadAgent 框架

5.3 多模态 agent

一文详解多模态智能体（LMAs）最新进展（核心组件/分类/评估/应用）

Large Multimodal Agents: A Survey

https://github.com/jun0wanan/awesome-large-multimodal-agents

5.4 OpenDevin

OpenDevin 出技术报告了，大模型 Agent 开发者必读

OpenDevin: An Open Platform for AI Software Developers as Generalist Agents

5.5 autogpt

GitHub 星标超 16 万，爆火 AutoGPT 进阶版来了：定制节点、多智能体协同

https://github.com/Significant-Gravitas/AutoGPT

5.6 DAAG

三「模」联盟，谷歌 DeepMind 缔造终身学习智能体！

Diffusion Augmented Agents: A Framework for Efficient Exploration and Transfer Learning

5.7 VARP

GPT-4o 能玩《黑神话》！精英怪胜率超人类，无强化学习纯大模型方案

Can VLMs Play Action Role-Playing Games? Take Black Myth Wukong as a Study Case

5.8 MARL

北大领衔，多智能体强化学习研究登上 Nature 子刊

Efficient and scalable reinforcement learning for large-scale network control

5.9 MMRole

与「李白」赏图赋诗，同「猴哥」直面天命，人大高瓴提出 MMRole 多模态角色扮演

MMRole: A Comprehensive Framework for Developing and Evaluating Multimodal Role-Playing Agents

https://github.com/YanqiDai/MMRole

5.10 Swarm

OpenAI 今天 Open 了一下：开源多智能体框架 Swarm

https://github.com/openai/swarm

5.11 agent-as-a-judge

卷起来！让智能体评估智能体，Meta 发布 Agent-as-a-Judge

Agent-as-a-Judge: Evaluate Agents with Agents

https://github.com/metauto-ai/agent-as-a-judge

5.12 Hammer

哪个模型擅长调用工具？这个 7B 模型跻身工具调用综合榜单第一

Hammer: Robust Function-Calling for On-Device Language Models via Function Masking

https://huggingface.co/MadeAgents

https://github.com/MadeAgents/Hammer

5.13 AgentOccam

不靠更复杂的策略，仅凭和大模型训练对齐，零样本零经验单 LLM 调用，成为网络任务智能体新 SOTA

AgentOccam: A Simple Yet Strong Baseline for LLM-Based Web Agents

5.14 苏格拉底学习

DeepMind 用语言游戏让大模型学 AlphaGo 自我博弈，数据限制不存在了

Boundless Socratic Learning with Language Games

5.15 从个人模拟到社会模拟

智能体模拟《西部世界》一样的社会，复旦大学等出了篇系统综述

From Individual to Society: A Survey on Social Simulation Driven by Large Language Model-based Agents

5.16 MetaGPT

ICLR2024

- MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework
- Data Interpreter: An LLM Agent For Data Science
- AFlow: Automating Agentic Workflow Generation

4w+ 的 stars 了

https://github.com/geekan/MetaGPT

5.17 insight-V

多智能体架构 Insight-V 来了！突破长链视觉推理瓶颈

Insight-V: Exploring Long-Chain Visual Reasoning with Multimodal Large Language Models

https://github.com/dongyh20/Insight-V

5.18 世界经济论坛的 agent 报告

Navigating the AI Frontier: A Primer on the Evolution and Impact of AI Agents

5.19 Claude: building effective agents

Claude 官方发布《Agent 构建指南》，附 PDF 下载

https://www.anthropic.com/research/building-effective-agents

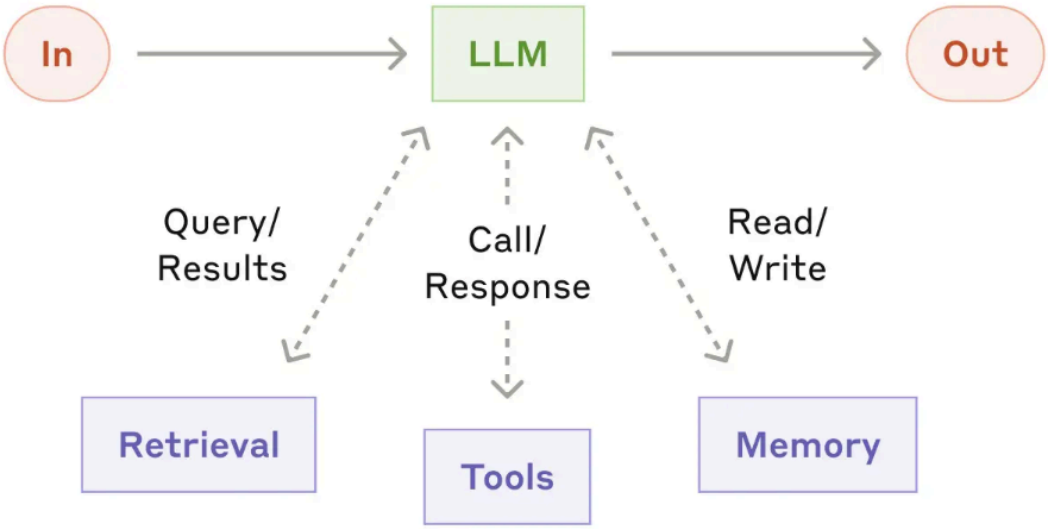
Agent 系统分为两大类：

- 工作流 (Workflows)：
  - 特点：通过预定义的代码路径来编排 LLM 和工具的系统。更像是一个精心设计的流程，每一步都清晰可控。

  - 场景：当任务非常明确，而且可以分解成一系列固定的步骤时，就像流水线上的工作一样，用“工作流程”就足够了。
- 智能体 (Agents)：
  - 特点：由 LLM 动态地指导自身流程和工具使用的系统。更像是一个自主的决策者，能够根据环境反馈灵活调整行动。
  - 场景：当任务需要很大的灵活性，而且需要模型自己做决策时，就像一个需要随机应变的指挥官，这时候“智能体”就更适合。

现有的框架：+ LangGraph（LangChain 的工具）：就像一套功能强大的乐高套件，可以用来搭建各种复杂的 Agent 系统。https://langchain-ai.github.io/langgraph/ + Amazon Bedrock 的 AI Agent 框架：就像一个专业的工具箱，提供了各种构建 Agent 系统的工具和组件。https://aws.amazon.com/cn/bedrock/agents/ + Rivet（拖放式 GUI LLM 工作流构建器）：就像一个可视化编辑器，可以通过拖拽的方式来构建 LLM 的工作流程，非常方便。https://rivet.ironcladapp.com/ + Vellum（复杂工作流的构建和测试工具）：就像一个高级的实验室，可以用来构建和测试复杂的工作流程。https://www.vellum.ai/

5.19.1 基石：augmented LLM

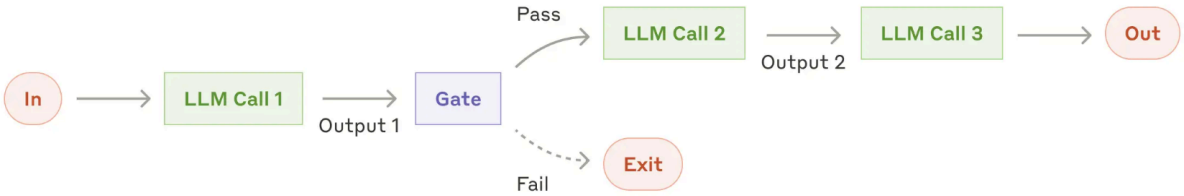


通过**检索**、**工具和记忆**等机制扩展大语言模型的能力，这样大语言模型能够主动运用这些能力来生成自己的搜索查询、选择合适的工具，并决定保留哪些信息。

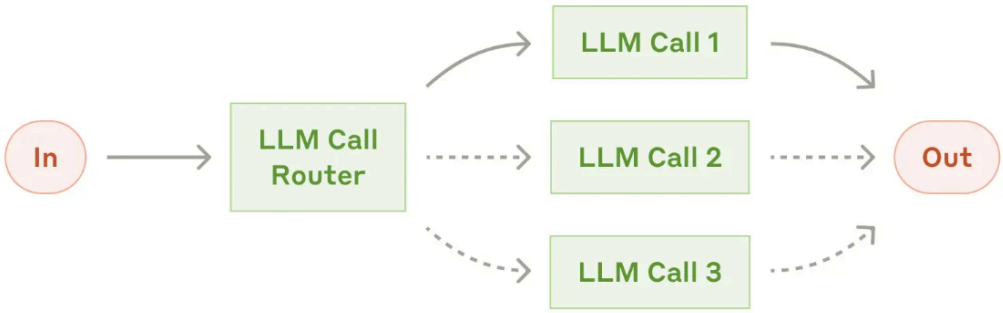
Anthropic 有一个上下文协议（Model Context Protocol，MCP），允许开发者通过简单的客户端实现与不断增长的第三方工具生态系统集成，参考https://www.anthropic.com/news/model-context-protocol

5.19.2 workflow

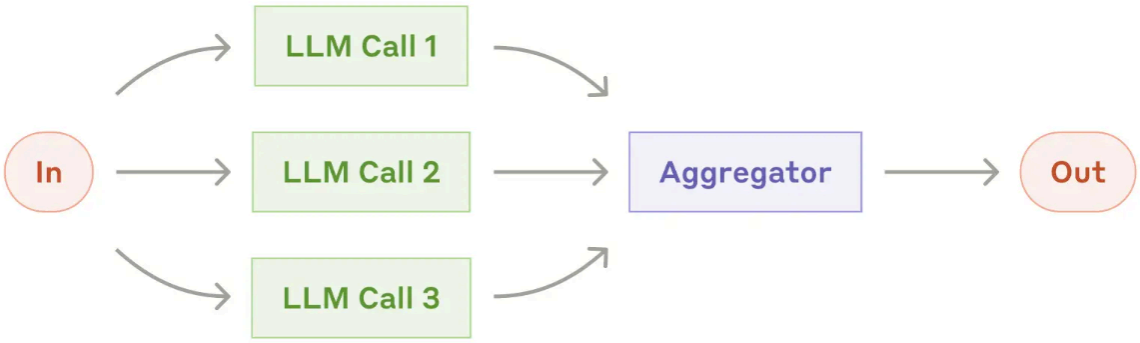
- prompt chaining：将一个任务分解成一系列步骤，其中的每个 LLM 的调用都会处理前一个调用的输出，可以在任何中间步骤中添加程序化的检查（见图中的“门控 Gate”），以确保流程仍在正轨上，即有一个失败就 exit



- Routing：将不同类型的客户服务查询（一般问题、退款请求、技术支持）导向不同的下游流程、提示和工具，例如将简单/常见的问题路由到较小的模型（如 Claude 3.5 Haiku），将困难/不常见的问题路由到功能更强大的模型（如 Claude 3.5 Sonnet），以优化成本和速度。

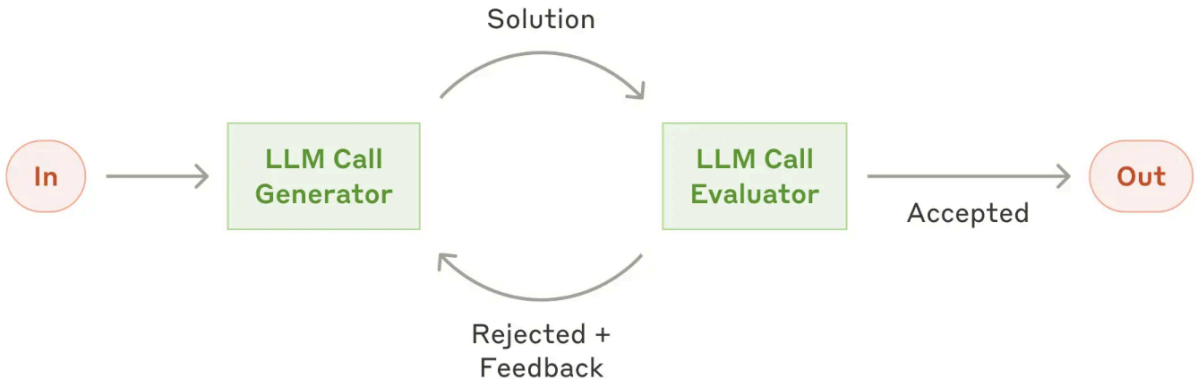


- Parallelization：同时执行多个任务，并通过程序化方式整合结果。适用：
  - 分段处理：
    - 构建安全防护机制，一个模型实例负责处理用户查询，而另一个模型实例负责筛选不当内容或请求。
    - 自动化评估模型性能，每个模型调用负责评估模型在给定提示下的不同性能指标。
  - 多重投票：
    - 对代码进行漏洞审查，多个不同的提示分别审查代码，并在发现问题时进行标记。
    - 评估内容是否不当，多个提示从不同角度进行评估，或采用不同的投票阈值来平衡误报和漏报。



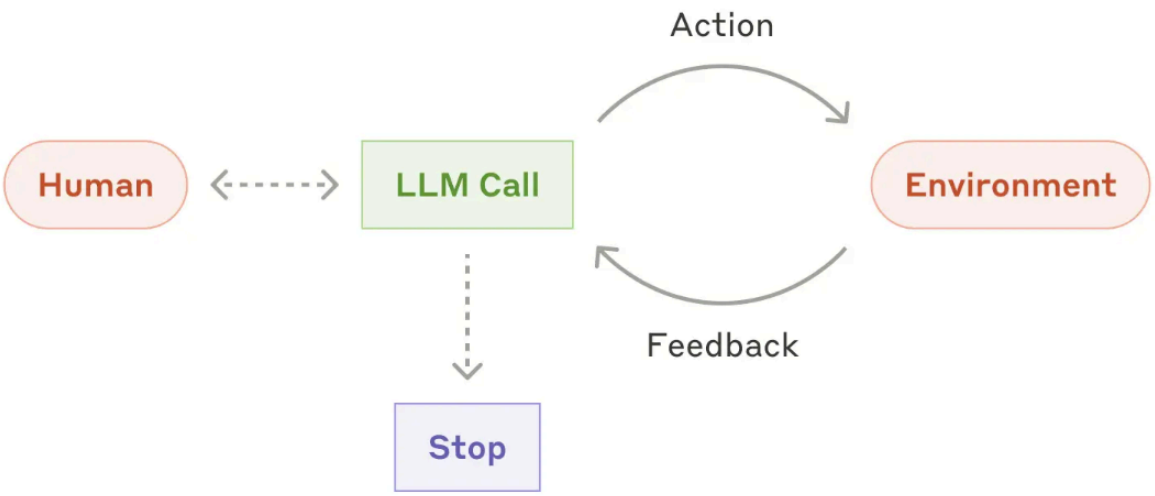
- Orchestrator-workers（协调者-工作者模式）：一个中央 LLM 会**动态地分解任务**，并将这些子任务分配给不同的工作者模型，最后再整合所有工作者的结果。适用：
  - 需要对**多个文件**进行复杂修改的编码产品。
  - 需要从**多个来源**收集并分析信息以寻找相关内容的搜索任务。

- Evaluator-optimizer：一个 LLM 负责调用**生成**响应，而另一个 LLM 调用则在一个**循环**中提供**评估和反馈**。适用：
  - 文学翻译，比如翻译模型**最初**可能无法捕捉到的细微差别，但评估器模型可以提供有用的评审意见。
  - 需要进行**多轮**搜索和分析以收集全面信息的复杂搜索任务，评估器可以用来决定是否需要进一步搜索。



5.19.3 Agents

当 LLM 在理解复杂输入、推理规划、可靠使用工具和从错误中恢复等关键能力上成熟时，智能体可以处理开放式问题，无需预先定义步骤，并能根据环境反馈自主决策 **。在特定节点或遇到困难时暂停的功能，以便引入人工干预或反馈。



6 一些其他比较重要的工作

6.1 几篇出现频率比较高的论文

Scaling instruction-finetuned language models 引用数 800+

How can we know what language models know? 引用数 800+

Chain of thought prompting elicits reasoning in large language models引用 1800+

6.2 Anthropic 的一些工作

Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback

Studying Large Language Model Generalization with Influence Functions

Measuring Faithfulness in Chain-of-Thought Reasoning

从 Claude 3 中提取数百万特征，首次详细理解大模型的「思维」

Scaling Dictionary Learning to Claude 3 Sonnet

LLM 惊现篡改代码获得奖励，欺骗人类无法根除逆转！Anthropic 新作揭露惊人真相

SYCOPHANCY TO SUBTERFUGE: INVESTIGATING REWARD TAMPERING IN LANGUAGE MODELS

7 个性化搜索

随便找一篇Denoising Attention for Query-aware User Modeling in Personalized Search，来看看它的参考文献：

学术界：

- A Transformer-based Embedding Model for Personalized Product Search，sigir20
- Learning a Fine-Grained Review-based Transformer Model for Personalized Product Search，sigir21
- RLPer: A Reinforcement Learning Model for Personalized Search，www20

工业界：

- A Zero Attention Model for Personalized Product Search，CIKM19，亚马逊

- Real-time Personalization using Embeddings for Search Ranking at Airbnb，KDD18，airbnb
- End-to-End Deep Attentive Personalized Item Retrieval for Online Content-sharing Platforms，www20，Google
- Towards Personalized and Semantic Retrieval: An End-to-End Solution for E-commerce Search via Embedding Learning，sigir20，京东
- Personalized Query Suggestions，sigir20，LinkedIn
- A GNN-based Multi-task Learning Framework for Personalized Video Search，WSDM22，百度

7.1 q-i 双塔的改进

7.1.1 引入 location+social

Embedding-based Retrieval in Facebook Search，KDD20

q-d 双塔结构，在两个塔的最底层均加入：

- location：用户所处地理位置，如城市
- social：facebook 是社交网络，通过另一个基于 graph 的模型训练得到的 user 和 item emb，直接加进来

7.1.2 引入用户行为序列

Encoding History with Context-aware Representation Learning for Personalized Search，sigir20，人大，提出 HTPS

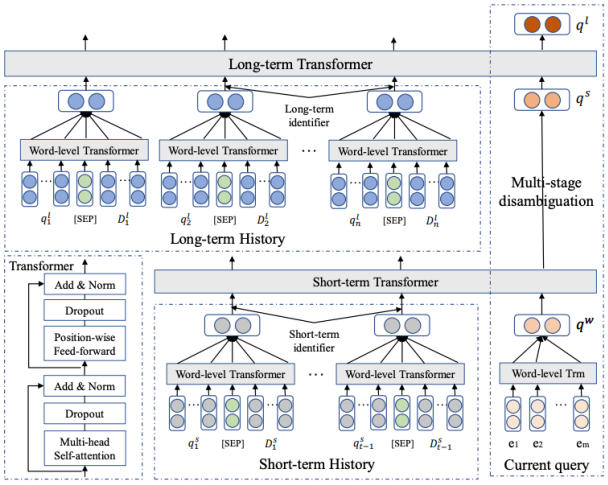


Figure 1: The architecture of the query disambiguation model. Given a new query, we encode the query terms, short-term history, and long-term history as contextual information separately with hierarchical transformer to disambiguate the current query .

把用户历史的 q-d pair 对和当前 query 一起，过短期 transformer 和长期 transformer 得到输出 q^l。

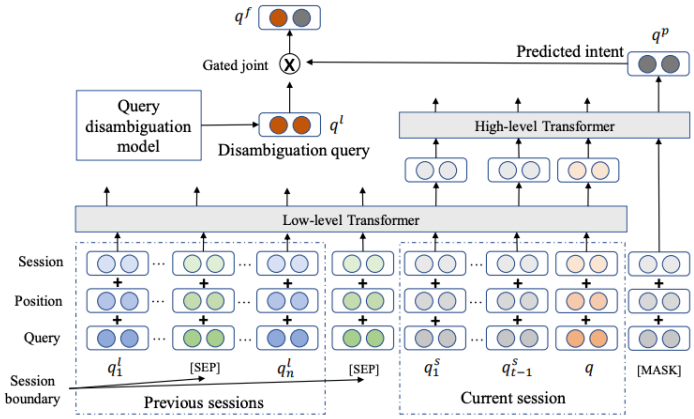


Figure 2: The architecture of the personalized language model, which is used to learn the user's personalized search pattern. We first enhance the queries in the current session based on historical patterns, and then summarize them to predict the current intent. Finally, we combine two models with a gate unit.

把 q^l 加上 [mask]，过 transoformer 得到预估的 intent q^p

然后将 q^l 和 q^p 通过 gate nn 融合得到最终的 context-aware 的 query 表示 q^f

最终 doc 和 query 的打分包括两部分，通过 ϕ（一个 MLP，激活是 tanh）进行融合：

$$p(d \mid q, H) = \phi\left(p(d, q), p\left(d, q^H\right)\right)$$

- $p(d, q)$：q 和 d 的语义相似度，可以用正常的 nlp 模型得到
- $p\left(d, q^H\right)$：q 和 d 的个性化得分，公式如下，其中 s^R 是 cos：

$$p\left(d, q^H\right) = \phi\left(s^R\left(q^s, d^w\right), s^R\left(q^l, d^w\right), s^R\left(q^p, d^w\right), s^R\left(q^f, d^w\right)\right)$$

有两个 loss：

- pred loss：预估 intent，即下一个 query，拿 q^p 与下一个 query 中各个词向量的 avg 算 cos
- rank loss：依据 $p(d \mid q, H)$ 算 lambda rank 的 pairwise loss

7.1.3 三塔 +gnn 邻居 +mtl

A GNN-based Multi-task Learning Framework for Personalized Video Search，WSDM22，百度，提出 MGNN-PVS

现有的 PSM(g personalized search methods) 大多使用用户反馈（如点击）进行训练，缺点：

- 反馈信号大部分表达的是吸引力而非相关性
- 用户的历史信号比较稀疏，很难学好 PSM

两张二部图：u-q 和 q-d

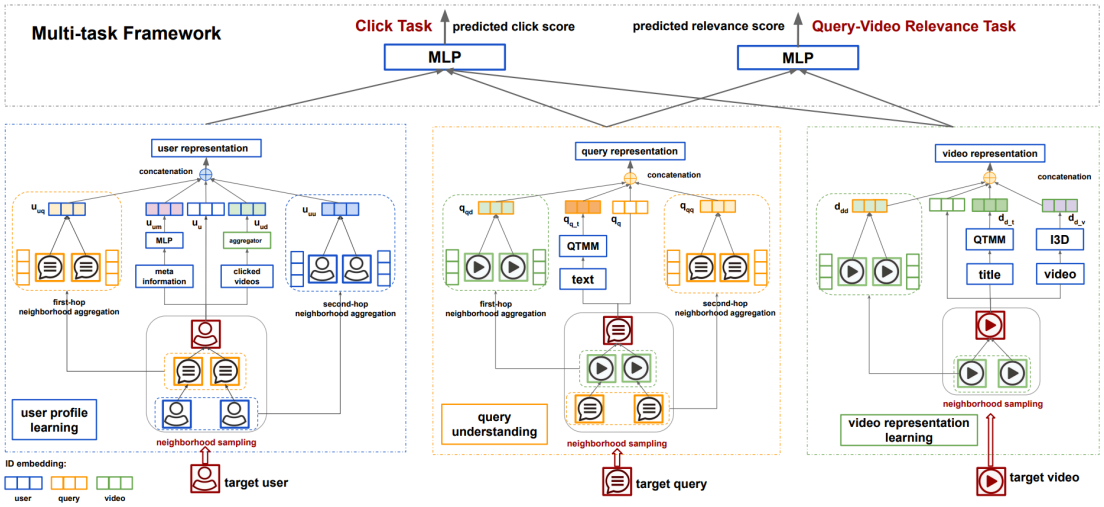


Figure 4: An illustration of our GNN-based multi-task framework. Given the triple <u_u, q_q, d_d>, we first apply the QTMM and I3D to learn the semantic representations of q_q and d_d. Then, we sample fixed-size neighbors for u_u, q_q, d_d from the u-q and q-d graphs and leverage the graph information with the proposed hierarchical GNN architecture simultaneously capturing both local and higher-order interactions among nodes to enhance their representation. Finally, we combine representations learned from text, video and graph for the click task and query-video relevance task for personalized video search.

3 个塔：

- user：
  - user 自己
  - 一跳邻居（u->q）的 q
  - 二跳邻居（u->q->u）的 u
- query：
  - query 自己
  - 一跳邻居（q->d）的 doc
  - 二跳邻居（q->d->q）的 query
- doc：
  - doc 自己的 title 向量（训练 query-正 title-负 title 的 triplet loss）和 video 向量（训练 video-正 query-负 query 的 triplet loss）
  - 二跳邻居（d->q->d）的 doc

两个 task：

- ctr 预估：u 和 q 拼一起过 nn 得到个性化的 q，再和 d 过 nn 得到的向量算内积，得到预估值，用交叉熵
- 相关性预估：q 过另一个 nn，d 过另一个 nn，内积，用 mse

8 RAG

Retrieval-Augmented Generation for Large Language Models: A Survey

RAG 全链路的关键模块解析

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Meta 提出全新文档级嵌入框架，利用 LLM 来增强信息检索能力

LLM-Augmented Retrieval: Enhancing Retrieval Models Through Language Models and Doc-Level Embedding

8.1 RankRAG

RAG 微调 Llama 3 竟超越 GPT-4！英伟达 GaTech 华人学者提出 RankRAG 框架

RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs

8.2 graphRAG

微软开源的 GraphRAG 爆火，Github Star 量破万，生成式 AI 进入知识图谱时代?

https://github.com/microsoft/graphrag

From Local to Global: A Graph RAG Approach to Query-Focused Summarization

8.3 RAGChecker

给 RAG 系统做一次全面「体检」，亚马逊开源 RAGChecker 诊断工具

RAGCHECKER: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation

https://github.com/amazon-science/RAGChecker

8.4 TAG

表格增强生成 TAG 登场：解锁 AI 自然语言与数据库的完美结合

8.5 Storm

斯坦福开源学术研究神器 STORM 再进化，AI 智能体像人一样进行圆桌讨论

Into the Unknown Unknowns: Engaged Human Learning through Participation in Language Model Agent Conversations

https://github.com/stanford-oval/storm

https://storm.genie.stanford.edu/

8.6 Block-atttention RAG

RAG 新突破：块状注意力机制实现超低延迟检索增强

Block-Attention for Efficient RAG

8.7 2024 rags

RAG 七十二式：2024 年度 RAG 清单

9 LLM 模型融合

https://github.com/arcee-ai/mergekit

SOLAR 10.7B: Scaling Large Language Models with Simple yet Effective Depth Up-Scaling

LLM 合并新思路：进化算法 + 零训练-> 新任务

Evolutionary Optimization of Model Merging Recipes

https://github.com/SakanaAI/evolutionary-model-merge

10 LLM auto-ml

10.1 LLaMA-NAS

用神经架构搜索给 LLM 瘦身，模型变小，准确度有时反而更高

LLaMA-NAS: Efficient Neural Architecture Search for Large Language Models

10.2 SELA

MetaGPT 开源 SELA，用 AI 设计 AI，效果超越 OpenAI 使用的 AIDE

SELA: Tree-Search Enhanced LLM Agents for Automated Machine Learning

https://github.com/geekan/MetaGPT/tree/main/metagpt/ext/sela

11 prompt engineering

万字长文总结提示词技巧！新加坡首届 GPT-4 提示工程大赛冠军最新分享 提出了 CO-STAR 框架

高能干货分享，有关提示词工程的一切都在这份教程里

https://github.com/NirDiamant/Prompt_Engineering

吴恩达：四个步骤，让大模型变得更好

Can Generalist Foundation Models Outcompete Special-Purpose Tuning? Case Study in Medicine

让 gpt4 生成 cot 和答案的模板

Self-generated Chain-of-thought Template

```
## Question: {{question}}
{{answer_choices}}
## Answer
```

model generated chain of thought explanation

Therefore, the answer is [*final model answer (e.g. A,B,C,D)*]

Algorithm 1 Algorithmic specification of Medprompt, corresponding to the visual representation of the strategy in Figure 4.

Input: Development data $\mathcal{D}$, Test question Q
Preprocessing:
for `each question` q `in` $\mathcal{D}$ **do**
  Get an embedding vector v_q for q.
  Generate a chain-of-thought C_q and an answer A_q with the LLM.
  if Answer A_q is correct **then**
    Store the embedding vector v_q, chain-of-thought C_q, and answer A_q.
  end if
end for

Inference Time:
Compute the embedding v_Q for the test question Q.
Select the 5 most similar examples $\{(v_{Q_i}, C_{Q_i}, A_{Q_i})\}_{i=1}^{5}$ from the preprocessed training data using KNN, with the distance function as the cosine similarity: $\text{dist}(v_q, v_Q) = 1 - \frac{\langle v_q, v_Q \rangle}{\|v_q\| \|v_Q\|}$.
Format the 5 examples as context $\mathcal{C}$ for the LLM.
for `5 times` **do**
  Shuffle the answer choices of the test question.
  Generate a chain-of-thought C_q^k and an answer A_q^k with the LLM and context $\mathcal{C}$.
end for
Compute the majority vote of the generated answers $\{A_q^k\}_{k=1}^{K}$:

$$A^{\text{Final}} = \text{mode}(\{A_q^k\}_{k=1}^{K}),$$

where $\text{mode}(X)$ denotes the most common element in the set X.
Output: Final answer A^{Final}.

看着是借助 GPT4+COT+RAG+ 投票

- 拿一坨 question 得到他们的向量，并按照上图的模板让 gpt 生成 COT 和答案，人工判断，对的存进知识库里
- 预测阶段：
  - 拿测试 question 的向量从知识库里查出 5 个最像 (cos 距离) 的 (q, cot, answer) 作为 context
  - 循环 5 次：shuffle 测试 question 的答案选项，让 LLM 回答
  - 对生成的答案投票，选票数最多的

11.1 APE

还在人工炼丹？自动提示工程指南来了，还带从头实现

https://github.com/marshmellow77/automated-prompt-engineering-from-scratch

11.2 PAS

还在死磕 AI 咒语？北大-百川搞了个自动提示工程系统 PAS

PAS: Data-Efficient Plug-and-Play Prompt Augmentation System

11.3 ell

OpenAI 前研究者发布提示词工程框架 ell，升级版 LangChain，支持版本控制和多模态

https://github.com/MadcowD/ell

12 可解释 AI

XAI 有什么用？探索 LLM 时代利用可解释性的 10 种策略

Usable XAI: 10 Strategies Towards Exploiting Explainability in the LLM Era

https://github.com/JacksonWuxs/UsableXAI_LLM

12.1 综述

可解释性终极追问，什么才是第一性解释？20 篇 CCF-A+ICLR 论文给你答案

12.2 TransformerLens

Neel Nanda（deepmind）的项目

https://transformerlensorg.github.io/TransformerLens/

12.3 ecco

https://www.eccox.io/

https://jalammar.github.io/explaining-transformers/

https://jalammar.github.io/hidden-states/

12.4 interpretability in the wild

Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small

https://github.com/redwoodresearch/Easy-Transformer

12.5 activation engineering

Activation Addition: Steering Language Models Without Optimization

12.6 representation engineering

Representation Engineering: A Top-Down Approach to AI Transparency

12.7 transformer-debugger

https://github.com/openai/transformer-debugger/tree/main

12.8 painter

八问八答搞懂 Transformer 内部运作原理

Transformer Layers as Painters

12.9 transformer explainer

黑匣子被打开了！能玩的 Transformer 可视化解释工具，本地运行 GPT-2、还可实时推理

TRANSFORMER EXPLAINER: Interactive Learning of Text-Generative Models

http://poloclub.github.io/transformer-explainer/

https://bbycroft.net/llm

12.10 superposition

Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet

12.11 3Blue1Brown

用最直观的动画，讲解 LLM 如何存储事实，3Blue1Brown 的这个视频又火了

https://www.youtube.com/watch?v=9-Jl0dxWQs8

12.12 Monitor

他们掰开神经元，终于让大模型 9.8 大于 9.11 了：神秘创业公司，开源 AI「洗脑」工具

https://transluce.org/observability-interface

https://monitor.transluce.org/dashboard/chat

13 Lifelong learning of LLM

(toread)

Towards Lifelong Learning of Large Language Models: A Survey

https://github.com/qianlima-lab/awesome-lifelong-learning-methods-for-llm

14 自省

LLM 比之前预想的更像人类，竟也能「三省吾身」

Looking Inward: Language Models Can Learn About Themselves by Introspection

15 具身智能

大模型走向物理世界，TeleAI 发布大模型驱动的具身智能综述，覆盖 300 篇文献

Embodied-AI with large models: research and challenges

15.1 ReKep

李飞飞团队提出 ReKep，让机器人具备空间智能，还能整合 GPT-4o

ReKep: Spatio-Temporal Reasoning of Relational Keypoint Constraints for Robotic Manipulation

https://github.com/huangwl18/ReKep

15.2 GR-2

GR-2 登场！ByteDance Research 提出机器人大模型，具备世界建模和强大泛化能力

GR-2: A Generative Video-Language-Action Model with Web-Scale Knowledge for Robot Manipulation

https://gr2-manipulation.github.io/

15.3 RDT-1B

清华开源全球最大双臂机器人扩散大模型 RDT，懂调酒能遛狗，登顶 HF 具身热榜

RDT-1B: a Diffusion Foundation Model for Bimanual Manipulation

15.4 HIL-SERL

强化学习训练一两个小时，100% 自主完成任务：机器人 ChatGPT 时刻真来了?

Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning

15.5 Genesis

历时 2 年，华人团队力作，震撼开源生成式物理引擎 Genesis，可模拟世界万物

https://github.com/Genesis-Embodied-AI/Genesis

15.6 language of motion

李飞飞团队统一动作与语言，新的多模态模型不仅超懂指令，还能读懂隐含情绪

The Language of Motion: Unifying Verbal and Non-verbal Language of 3D Human Motion

16 语音

16.1 Mini-Omni

让大模型能听会说，国内机构开源全球首个端到端语音对话模型 Mini-Omni

Mini-Omni: Language Models Can Hear, Talk While Thinking in Streaming

https://github.com/gpt-omni/mini-omni

16.2 Illuminate

任意论文一键变播客，谷歌正式发布 Illuminate，它能重构研究者的学习方式吗?

https://illuminate.google.com/home

17 LLM+ 芯片设计

登上 Nature 的 AI 芯片设计屡遭质疑，谷歌发文反击，Jeff Dean：质疑者连预训练都没做

That Chip Has Sailed: A Critique of Unfounded Skepticism Around AI for Chip Design

18 其他

18.1 安全性

Anthropic 安全负责人：在超级 AI「毁灭」人类之前，我们可以做这些准备

OpenAI 最新 53 页论文：ChatGPT 看人下菜碟，对“小美”比“小帅”更友好

First-Person Fairness in Chatbots

翁荔 B 站分享原文：AI 安全与“培养”之道

18.2 time-LLM

谁说大象不能起舞！重编程大语言模型实现跨模态交互的时序预测 | ICLR 2024

Time-LLM: Time Series Forecasting by Reprogramming Large Language Models

https://github.com/KimMeen/Time-LLM

将**时序预测任务**转换成一个可以由 LLMs 有效解决的**语言任务**，成功激活了 llm 做**高精度时序推理**的能力。

- 时序输入重编程
- 提示做前缀

18.3 长尾

A Systematic Review on Long-Tailed Learning

18.4 文本匹配效果还行的模型

大多是基于 sentence-bert 的，m3e-base 在电商语料上试过，效果不错

https://huggingface.co/moka-ai/m3e-base

https://huggingface.co/shibing624/text2vec-base-chinese

18.5 本地知识库

https://github.com/chatchat-space/Langchain-Chatchat

18.6 llm 应用合辑

- ChatGPT 聚合站：https://hokex.com
- 游戏生成站：https://latitude.io/
- 家庭作业辅助站：https://ontimeai.com/
- 文字转语音站：https://www.resemble.ai/
- 艺术作画站：https://starryai.com/
- logo 制作站：https://www.logoai.com/
- ai 写作站：https://www.getconch.ai/
- 音乐制作站：https://soundraw.io/
- 声音模拟站：https://fakeyou.com/
- 一句话生成一段视频：https://runwayml.com/
- 文字转语音：https://murf.ai/

18.7 swiftsage

大语言模型在开放世界中的推理能力探索实践

SwiftSage: A Generative Agent with Fast and Slow Thinking for Complex Interactive Tasks

18.8 达摩院大模型技术交流

https://developer.aliyun.com/live/248332

ppt：链接 密码：5yyf

18.9 回译

通过单语数据提升 NMT 模型最高效的方法之一是回译（back-translation）。如果我们的目标是训练一个英语到德语的翻译模型，那么可以首先训练一个从德语到英语的翻译模型，并利用该模型翻译所有的单语德语数据。然后基于原始的英语到德语数据，再加上新生成的数据，我们就能训练一个英语到德语的最终模型。

Understanding Back-Translation at Scale

18.10 nan 问题

解决 pytorch 半精度 amp 训练 nan 问题

18.11 时间序列

LLM 用于时序预测真的不行，连推理能力都没用到

Are Language Models Actually Useful for Time Series Forecasting?

18.12 人脑

MIT 大牛新作震惊学界！AI「长脑子」了？LLM 惊现「人类脑叶」结构并有数学代码分区

The Geometry of Concepts: Sparse Autoencoder Feature Structure

18.13 LLM for 算法设计

调研 180 多篇论文，这篇综述终于把大模型做算法设计理清了

A Systematic Survey on Large Language Models for Algorithm Design

18.14 深度生成模型课程

教授何恺明在 MIT 的第二门课——《深度生成模型》，讲座 PPT 陆续已出

https://mit-6s978.github.io/schedule.html

https://mit-6s978.github.io/assets/pdfs/lec1_intro.pdf

https://mit-6s978.github.io/assets/pdfs/lec2_vae.pdf

https://mit-6s978.github.io/assets/pdfs/lec3_ar.pdf

https://mit-6s978.github.io/assets/pdfs/lec4_gan.pdf

https://mit-6s978.github.io/assets/pdfs/lec5_diffusion.pdf

18.15 其他

小模型性能饱和、表现不佳，根源是因为 Softmax?

Why do small language models underperform? Studying LM Saturation via the Softmax Bottleneck

Ilya Sutskever 的推荐清单

传说中 Ilya Sutskever 精选论文清单：AI 领域 40 大论文完整版「破解」完成

2024 年大模型 LLM 还有哪些可研究的方向?

Hinton 万字访谈：用更大模型「预测下一个词」值得全力以赴

ChatGPT 如何「思考」? 心理学和神经科学破解 AI 大模型，Nature 发文

适应多形态多任务，最强开源机器人学习系统「八爪鱼」诞生

Octo: An Open-Source Generalist Robot Policy

LeCun 新作：神经网络在实践中的灵活性到底有多大?

Just How Flexible are Neural Networks in Practice?

清华包揽最佳论文 + 时间检验奖，山大获荣誉提名，SIGIR 2024 奖项出炉

19 一些记录

19.1 打印模型参数量

https://stackoverflow.com/questions/49201236/check-the-total-number-of-parameters-in-a-pytorch-model

```
pytorch_total_params = sum(p.numel() for p in model.parameters())

pytorch_total_trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

# Load the model
from transformers import BartForConditionalGeneration
from transformers import T5ForConditionalGeneration
def cal(model):
  pytorch_total_trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
  return pytorch_total_trainable_params

model = BartForConditionalGeneration.from_pretrained("facebook/bart-base")
print("bart-base")
print(cal(model)) # 6L 139420416 139M

model = BartForConditionalGeneration.from_pretrained("facebook/bart-base")
print("bart-base")
print(cal(model)) # 12L 406291456 406M

model = T5ForConditionalGeneration.from_pretrained("t5-small")
print("t5-small")
print(cal(model)) # 6L 60506624 65M

model = T5ForConditionalGeneration.from_pretrained("t5-base")
print("t5-base")
print(cal(model)) # 12L 222903552 223M


model = T5ForConditionalGeneration.from_pretrained("t5-large")
print("t5-large")
print(cal(model)) # 24L 737668096 738M
```

19.2 往现有 tokenizer 里加一些特殊 token

https://stackoverflow.com/questions/69191305/how-to-add-new-special-token-to-the-tokenizer

```
num_added_toks = tokenizer.add_tokens(['[EOT]'], special_tokens=True) ##This line is updated
model.resize_token_embeddings(len(tokenizer))

###The tokenizer has to be saved if it has to be reused
tokenizer.save_pretrained(<output_dir>)
```

示例

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

print("Before")
print(tokenizer.all_special_tokens) # --> ['[UNK]', '[SEP]', '[PAD]', '[CLS]', '[MASK]']
```

```
print(tokenizer.all_special_ids)    # --> [100, 102, 0, 101, 103]


special_tokens_dict = {'additional_special_tokens': ['[EOT]']}
num_added_toks = tokenizer.add_special_tokens(special_tokens_dict)
# model.resize_token_embeddings(len(tokenizer))  # --> Embedding(30523, 768)

tok_id = tokenizer.convert_tokens_to_ids('[EOT]')  # --> 30522

print("After")
print(tokenizer.all_special_tokens) # --> ['[UNK]', '[SEP]', '[PAD]', '[CLS]', '[MASK]']
print(tokenizer.all_special_ids)    # --> [100, 102, 0, 101, 103]
```

19.3 python 的读写锁

一个写，多个并行读：https://pypi.org/project/readerwriterlock/

19.4 pytorch 的显存泄露

https://github.com/pytorch/pytorch/issues/13246#issuecomment-445770039

19.5 torch profiling

https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html

可以拿这个来可视化：https://ui.perfetto.dev/

- 点击 open trace file 上传 json 文件
- timeline 中有两个 python 进程，点击 cuda kernel 会出现箭头，方便找到是**哪个 op 调用了该 kernel**
  - 靠上的 python 进程是 host 侧进程（主要是用户代码中调用的一些 **API/pytorch op**，能比较方便能和训练代码对应上）
  - 靠下的 python 进程是 device（gpu）侧进程（记录实际 cuda kernel 的执行和一些性能相关的数据）

device timeline 比较稀疏的情况下训练性能较差，GPU 利用率较低，可能需要排查下训练代码是否有问题

19.6 显存泄露排查

https://pytorch.ac.cn/docs/stable/torch_cuda_memory.html

https://pytorch.org/blog/understanding-gpu-memory-1/

https://pytorch.org/blog/understanding-gpu-memory-2/

检查显存

```
# (c) Meta Platforms, Inc. and affiliates.
# https://pytorch.org/blog/understanding-gpu-memory-1/
import logging
import socket
from datetime import datetime, timedelta

import torch

from torchvision import models

logging.basicConfig(
   format="%(levelname)s:%(asctime)s %(message)s",
   level=logging.INFO,
```

```
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger: logging.Logger = logging.getLogger(__name__)
logger.setLevel(level=logging.INFO)

TIME_FORMAT_STR: str = "%b_%d_%H_%M_%S"

# Keep a max of 100,000 alloc/free events in the recorded history
# leading up to the snapshot.
MAX_NUM_OF_MEM_EVENTS_PER_SNAPSHOT: int = 100000

def start_record_memory_history() -> None:
    if not torch.cuda.is_available():
        logger.info("CUDA unavailable. Not recording memory history")
        return

    logger.info("Starting snapshot record_memory_history")
    torch.cuda.memory._record_memory_history(
        max_entries=MAX_NUM_OF_MEM_EVENTS_PER_SNAPSHOT
    )

def stop_record_memory_history() -> None:
    if not torch.cuda.is_available():
        logger.info("CUDA unavailable. Not recording memory history")
        return

    logger.info("Stopping snapshot record_memory_history")
    torch.cuda.memory._record_memory_history(enabled=None)

def export_memory_snapshot() -> None:
    if not torch.cuda.is_available():
        logger.info("CUDA unavailable. Not exporting memory snapshot")
        return

    # Prefix for file names.
    host_name = socket.gethostname()
    timestamp = datetime.now().strftime(TIME_FORMAT_STR)
    file_prefix = f"{host_name}_{timestamp}"

    try:
        logger.info(f"Saving snapshot to local file: {file_prefix}.pickle")
        torch.cuda.memory._dump_snapshot(f"{file_prefix}.pickle")
    except Exception as e:
        logger.error(f"Failed to capture memory snapshot {e}")
        return

# Simple Resnet50 example to demonstrate how to capture memory visuals.
def run_resnet50(num_iters=5, device="cuda:0"):
    model = models.resnet50().to(device=device)
    inputs = torch.randn(1, 3, 224, 224, device=device)
    labels = torch.rand_like(model(inputs))
    optimizer = torch.optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)
    loss_fn = torch.nn.CrossEntropyLoss()
```

```
    # Start recording memory snapshot history
    start_record_memory_history()

    for _ in range(num_iters):
        pred = model(inputs)
        loss_fn(pred, labels).backward()
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)

    # Create the memory snapshot file
    export_memory_snapshot()

    # Stop recording memory snapshot history
    stop_record_memory_history()

if __name__ == "__main__":
     # Run the resnet50 model
     run_resnet50()
```

同时 profile cpu 和显存

```
# (c) Meta Platforms, Inc. and affiliates.
# https://pytorch.org/blog/understanding-gpu-memory-1/
import logging
import socket
from datetime import datetime, timedelta

import torch

from torch.autograd.profiler import record_function
from torchvision import models

logging.basicConfig(
    format="%(levelname)s:%(asctime)s %(message)s",
    level=logging.INFO,
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger: logging.Logger = logging.getLogger(__name__)
logger.setLevel(level=logging.INFO)

TIME_FORMAT_STR: str = "%b_%d_%H_%M_%S"

def trace_handler(prof: torch.profiler.profile):
    # Prefix for file names.
    host_name = socket.gethostname()
    timestamp = datetime.now().strftime(TIME_FORMAT_STR)
    file_prefix = f"{host_name}_{timestamp}"

    # Construct the trace file.
    prof.export_chrome_trace(f"{file_prefix}.json.gz")

    # Construct the memory timeline file.
    prof.export_memory_timeline(f"{file_prefix}.html", device="cuda:0")
```

```python
def run_resnet50(num_iters=5, device="cuda:0"):
   model = models.resnet50().to(device=device)
   inputs = torch.randn(1, 3, 224, 224, device=device)
   labels = torch.rand_like(model(inputs))
   optimizer = torch.optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)
   loss_fn = torch.nn.CrossEntropyLoss()

   with torch.profiler.profile(
       activities=[
           torch.profiler.ProfilerActivity.CPU,
           torch.profiler.ProfilerActivity.CUDA,
       ],
       schedule=torch.profiler.schedule(wait=0, warmup=0, active=6, repeat=1),
       record_shapes=True,
       profile_memory=True,
       with_stack=True,
       on_trace_ready=trace_handler,
   ) as prof:
       for _ in range(num_iters):
           prof.step()
           with record_function("## forward ##"):
               pred = model(inputs)

           with record_function("## backward ##"):
               loss_fn(pred, labels).backward()

           with record_function("## optimizer ##"):
               optimizer.step()
               optimizer.zero_grad(set_to_none=True)

if __name__ == "__main__":
    # Warm up
    run_resnet50()
    # Run the resnet50 model
    run_resnet50()
```

19.7 各型号 gpu 对比

https://zhuanlan.zhihu.com/p/441153412

19.8 查看 python 的栈

```
pip install py-spy
py-spy dump --pid 1199
```

打出来:

```
Process 1199: /usr/bin/python3.10 -u torch_main.py
Python v3.10.14 (/usr/bin/python3.10)

Thread 0x7F62A2C43740 (active): "MainThread"
    _wait_for_tstate_lock (threading.py:1116)
    join (threading.py:1096)
    main (torch_main.py:776)
```

```
    <module> (torch_main.py:785)
Thread 0xAABBBCC (idle): "Thread-1"
    wait (threading.py:324)
    wait (threading.py:607)
    run (threading.py:1376)
    _bootstrap_inner (threading.py:1016)
    _bootstrap (threading.py:973)
Thread 0xAAAAA (idle): "Thread-3 (process)"
    wait (threading.py:320)
    get (queue.py:171)
    process (abase_writer.py:73)
    run (threading.py:953)
    _bootstrap_inner (threading.py:1016)
    _bootstrap (threading.py:973)
Thread 0xA992ACDA (idle): "Thread-4 (process)"
    wait (threading.py:320)
    get (queue.py:171)
    process (abase_writer.py:73)
    run (threading.py:953)
    _bootstrap_inner (threading.py:1016)
    _bootstrap (threading.py:973)
Thread 0xAFF11AA (active): "Thread-5 (read_file)"
    get_seq (ecom_seq_reader.py:200)
    read_file (torch_main.py:494)
    run (threading.py:953)
    _bootstrap_inner (threading.py:1016)
    _bootstrap (threading.py:973)
Thread 0x9922BCDA (idle): "Thread-6"
    wait (threading.py:324)
    wait (threading.py:607)
    run (tqdm/_monitor.py:60)
    _bootstrap_inner (threading.py:1016)
    _bootstrap (threading.py:973)
```

19.9 国内的 huggingface 模型下载地址

https://hf-mirror.com/

19.10 一些报错的解法

19.10.1 flash-attention2

https://github.com/Dao-AILab/flash-attention/issues/451

```
FLASH_ATTENTION_FORCE_BUILD=TRUE pip install flash-attn
```

20 GPU 机型对比

- L20：https://www.techpowerup.com/gpu-specs/l20.c4206
  - T flops：
    - tf32：59.8
    - fp32：59.8
    - bf16：119.5
    - fp16：119.5

- A800-40G: https://www.techpowerup.com/gpu-specs/a800-pcie-40-gb.c3964
  - T flops:
    - tf32: 156
    - fp32: 19.5
    - bf16: 312
    - fp16: 77.97